MODULE 26 · DOMAIN 4 - CLOUD NATIVE ARCHITECTURE

Observability

Metrics, logs and traces: Prometheus, Grafana, OpenTelemetry, Jaeger, Fluent Bit, and SLOs.

DOMAIN 4 of 4 EXAM WEIGHT 12% MODULE 26 of 35

admin@sthithapragna.com

Monitoring tells you **whether** something is wrong. Observability lets you ask **why**.

The three signals

SIGNAL	IS	BEST FOR
Metrics	Numbers sampled over time	Dashboards, trends, alerts. Cheap to keep
Logs	Timestamped records of events	Detail about what happened, and debugging
Traces	One request's path through every service	Finding which service made a request slow

> ON THE EXAM
>
> Metrics say something is wrong; traces say where; logs say what exactly happened.

Prometheus

- **The CNCF's second graduated project,** after Kubernetes itself.
- **Pulls metrics:** scrapes each target's /metrics endpoint over HTTP, on an interval.
- **Stores time series** in its own database, and queries them with PromQL.
- **Finds targets automatically** through Kubernetes service discovery.

> ON THE EXAM
>
> Prometheus pulls. That single design choice explains most Prometheus exam questions.

How the Prometheus stack fits together

01
Targets
Apps and exporters expose /metrics.

02
Scrape
Prometheus pulls every 15 to 60 seconds.

03
Store
Time series database, queried with PromQL.

04
Alert
Rules fire; Alertmanager groups, routes, silences.

05
Visualise
Grafana dashboards query Prometheus.

ON THE EXAM

Prometheus decides that an alert fires. Alertmanager decides who hears about it.

Pull versus push

PULL (PROMETHEUS'S DEFAULT)

Prometheus scrapes

- Prometheus controls timing
- A failed scrape shows the target is down
- Targets need no idea where Prometheus is
- Suits long-running services

PUSH

The job sends

- The workload sends its own metrics
- Needed when the job ends before a scrape
- Pushgateway holds them for scraping
- Also how OTLP senders work

THE TIEBREAKER

A batch job that finishes before the next scrape pushes to the Pushgateway.

The four metric types

TYPE	BEHAVES LIKE	EXAMPLE
Counter	Only ever goes up; resets to zero on restart	http_requests_total
Gauge	Goes up and down	memory in use, queue length
Histogram	Counts observations into buckets	request duration buckets
Summary	Pre-calculated quantiles in the client	request duration, 0.95 quantile

ON THE EXAM

Never graph a raw counter. Use rate() to turn it into 'per second'.

PromQL you should be able to read

```
1 # requests per second over the last 5 minutes, per service
2 sum by (service) (rate(http_requests_total[5m]))
3
4 # share of booking API requests that fail with a 5xx code
5 sum(rate(http_requests_total{service="booking-api",code=~"5.."}[5m]))
6   / sum(rate(http_requests_total{service="booking-api"}[5m]))
7
8 # 95th percentile latency, calculated from a histogram
9 histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

2 rate() over a counter, then summed per service.

5-6 Errors divided by all requests: the error ratio.

9 A percentile from histogram buckets, grouped by le.

Where Kubernetes metrics come from

SOURCE	PROVIDES
`node_exporter`	Node CPU, memory, disk and network; runs as a DaemonSet
`kube-state-metrics`	Object state: desired versus available replicas, Pod phases
`cAdvisor, via the kubelet`	Per-container CPU and memory use
`Component /metrics`	The API server, scheduler, etcd and others, in Prometheus format
`metrics-server`	Current CPU and memory, for kubectl top and autoscalers

ON THE EXAM

Usage from cAdvisor; object state from kube-state-metrics. Different questions, different sources.

metrics-server versus Prometheus

METRICS-SERVER

Right now, CPU and memory

- Current values only, held in memory
- CPU and memory only
- Serves the metrics API: metrics.k8s.io
- Feeds kubectl top and autoscalers

PROMETHEUS

Anything, over time

- Stores history
- Any metric at all
- Queries with PromQL, alerts with rules
- Feeds dashboards and alerting

THE TIEBREAKER

kubectl top or HPA not working: metrics-server. History, dashboards or alerts: Prometheus.

Around Prometheus

- **Alertmanager** groups, deduplicates, routes and silences alerts.
- **Grafana** draws dashboards from Prometheus, logs and traces. A Grafana Labs project, not CNCF.
- **Thanos and Cortex,** CNCF projects, add long-term storage and high availability.
- **The Prometheus Operator** manages Prometheus with custom resources, from module 4.

> ON THE EXAM
>
> Grafana visualises; it does not collect. Prometheus collects and stores.

Logs die with the Pod. Ship them off the node before they disappear.

Three ways to collect logs

PATTERN	HOW	WHEN
Node agent	A DaemonSet reads every container's log files on its node	The default choice. One agent per node
Sidecar	A shipper container beside the app	The app writes to files, not stdout
Direct	The app sends logs straight to the backend	Rarely; ties the app to the backend

> ON THE EXAM
>
> Apps log to stdout; a node-level agent ships everything. That is the standard pattern.

Logging tools

- **Fluentd:** a CNCF graduated log collector and router, with many plugins.
- **Fluent Bit:** its lightweight sibling, ideal as the node agent. Part of the same CNCF project.
- **Storage and search:** Loki, Elasticsearch or OpenSearch, or a cloud service.
- **Structured JSON logs** make searching and filtering far easier.

> ON THE EXAM
>
> Fluentd and Fluent Bit collect and forward. They are not the store you search.

A trace follows **one request** through every service it touches.

How tracing works

- **A span** is one operation, with a start time, a duration and attributes.
- **A trace** is the tree of spans for one request, sharing one trace ID.
- **Context propagation:** each service passes the trace ID on, in a header such as traceparent.
- **Every service must propagate it,** or the trace breaks into pieces.
- **Sampling** keeps a share of traces, to control cost.

> ON THE EXAM
>
> Tracing needs instrumentation in every hop. One service that drops the header breaks the chain.

OpenTelemetry

- **Vendor-neutral APIs, SDKs and a protocol, OTLP,** for traces, metrics and logs.
- **Instrument once,** send to any backend: Prometheus, Jaeger, Loki, or a commercial vendor.
- **The Collector** receives, processes and exports telemetry.
- **Formed from the merger** of OpenTracing and OpenCensus. Graduated in the CNCF, May 2026.

> ON THE EXAM
>
> 'Vendor-neutral instrumentation for all three signals': OpenTelemetry.

An OpenTelemetry pipeline

01
Instrument
SDK or auto-instrumentation in each service.

›

02
Send
Telemetry sent over OTLP.

›

03
Receive
The Collector's receivers accept it.

›

04
Process
Batch, sample, add attributes, redact.

›

05
Export
To Prometheus, Jaeger, Loki or a vendor.

ON THE EXAM

The Collector decouples apps from backends: change the backend without touching the code.

Currency check: the observability landscape

- **OpenTelemetry graduated** on 21 May 2026. Banks and guides that say incubating are dated.
- **OpenTracing is archived.** The official curriculum text still names it; teach OpenTelemetry.
- **Jaeger v2** is built on the OpenTelemetry Collector. Jaeger is CNCF graduated.
- **Prometheus 3** can ingest OpenTelemetry metrics sent over OTLP.

> ON THE EXAM
>
> A question offering OpenTracing as the tracing standard today is testing whether you know it was superseded.

SLIs, SLOs, SLAs and error budgets

TERM	IS	QUAYSIDE EXAMPLE
SLI	A measurement of service	Share of bookings answered in under 300 ms
SLO	A target for an SLI	99.9% of bookings under 300 ms, over 30 days
SLA	A contract, with consequences if missed	Refunds if availability falls below 99.5%
Error budget	100% minus the SLO	0.1% of bookings may be slow: spend it on releases

ON THE EXAM

Indicator measures, objective targets, agreement promises. SLOs are internal; SLAs are contractual.

What to measure

METHOD	MEASURES	FOR
Four golden signals	Latency, traffic, errors, saturation	Any user-facing service
RED	Rate, errors, duration	Request-driven services
USE	Utilisation, saturation, errors	Resources: CPU, memory, disks

ON THE EXAM

RED for services, USE for resources, the four golden signals for both.

> Quayside's nightly manifest export runs as a CronJob and finishes in about 40 seconds. Prometheus scrapes every 60 seconds.
>
> The team wants the job's duration and record count stored in Prometheus after every run.

What should the job do?

A Nothing; lower the scrape interval to one second

B Push its metrics to a Prometheus Pushgateway

C Report its numbers through metrics-server

D Print them so kubectl top can read them

◆ PAUSE AND DECIDE

What should the job do?

A Nothing; lower the scrape interval to one second

Expensive everywhere, and still races the job's end.

B Push its metrics to a Prometheus Pushgateway

Built for short jobs: it holds the metrics for scraping.

C Report its numbers through metrics-server

Serves only current CPU and memory, not custom metrics.

D Print them so kubectl top can read them

kubectl top shows resource usage, not job output.

THE SIGNAL

A job that ends before the scrape: Pushgateway.

> Some bookings take four seconds instead of half a second. Each request passes through the web front end, the booking API, the payment worker and two other services.
>
> Dashboards show higher overall latency, but not which service causes it.

What will most directly show where the time goes?

A. A dashboard of every node's CPU usage

B. Searching each service's logs for errors

C. kubectl top pods across the namespace

D. Tracing with OpenTelemetry and Jaeger

◆ PAUSE AND DECIDE

What will most directly show where the time goes?

A. A dashboard of every node's CPU usage
Shows load, not which call in the request was slow.

B. Searching each service's logs for errors
Slow is not an error, and logs are not linked per request.

C. kubectl top pods across the namespace
Current resource use only; no view of one request.

D. Tracing with OpenTelemetry and Jaeger
Spans show exactly which service used the time.

THE SIGNAL

'Which service in the chain is slow?': traces.

> A new cluster already runs Prometheus and Grafana. But kubectl top pods fails with: Metrics API not available.
>
> An autoscaler based on CPU cannot read any CPU figures either.

What is missing?

A. A Grafana dashboard for Pod CPU

B. A Jaeger backend for traces

C. metrics-server, for the metrics API

D. An Alertmanager route for CPU alerts

◆ PAUSE AND DECIDE

What is missing?

A A Grafana dashboard for Pod CPU

Dashboards display data; they do not serve the metrics API.

B A Jaeger backend for traces

Traces have nothing to do with CPU figures.

C metrics-server, for the metrics API

kubectl top and CPU autoscaling read metrics.k8s.io.

D An Alertmanager route for CPU alerts

Routes alerts; provides no metrics.

THE SIGNAL

'Metrics API not available': install metrics-server. Prometheus alone does not provide it.

> An old study guide calls OpenTracing the CNCF standard for tracing.
>
> Quayside must choose one vendor-neutral way to instrument its services for traces, metrics and logs, and to switch backends later.

Which should Quayside adopt?

A OpenTelemetry

B OpenTracing

C Jaeger's own client libraries

D Prometheus client libraries alone

◆ PAUSE AND DECIDE

Which should Quayside adopt?

A OpenTelemetry

All three signals, vendor-neutral, graduated; the Collector swaps backends.

B OpenTracing

Archived, and merged into OpenTelemetry. Traces only.

C Jaeger's own client libraries

Retired in favour of OpenTelemetry; ties you to one backend.

D Prometheus client libraries alone

Metrics only: no traces or logs.

THE SIGNAL

Vendor-neutral, all three signals: OpenTelemetry. OpenTracing is its archived ancestor.

Module 26 in five lines

- **Metrics** say something is wrong, **traces** say where, **logs** say what.
- **Prometheus pulls**; short jobs push to the **Pushgateway**; **Alertmanager** routes alerts.
- Counter, gauge, histogram, summary. **rate()** on counters. **metrics-server** is for top and autoscaling.
- **Fluent Bit** DaemonSets ship logs; **OpenTelemetry** (graduated 2026) instruments; **Jaeger** stores traces.
- **SLI** measures, **SLO** targets, **SLA** promises; the error budget is 100% minus the SLO.

Metrics also drive scaling. Next: **cloud native principles and autoscaling**.